

UniVerse

A Campus Companion

Routine (Timetable Viewer)

How students and teachers see their weekly class schedule from one shared table.

Leading University, Sylhet - Dept. of CSE

Course CSE-3240 (Project I) - Team Sherlocked

Flutter (Dart) - Supabase - Firebase FCM - OR-Tools CP-SAT

Owner: Robi (student/teacher features)

Key files: features/routine/*, core/models/routine_model.dart

1. What it does

The Routine screen shows a weekly class schedule with a day selector. The same screen + controller serves BOTH students and teachers - it is **role-aware**. Students see their cohort's classes; teachers see the classes they teach. Both read from one `routines` table.

file	job
<code>routine_service.dart</code>	The only Supabase layer. Two read paths over `routines`.
<code>routine_controller.dart</code>	Role-aware state: loads the right rows, tracks selected day, exposes that day's entries.

`routine_model.dart` Typed row + all the time/status helpers. Pure Dart, no Supabase.
(`RoutineEntry`)

2. Two read paths, one table

```
// student: filter by cohort
fetchForStudent(batch, section) -> WHERE batch=.. AND section=.. AND is_active

// teacher: filter by their code
fetchForTeacher(teacherCode) -> WHERE teacher_code=.. AND is_active
```

Why no semester filter for students? A batch+section cohort is only ever in one semester at a time, so batch+section already uniquely identifies their routine. Filtering by semester too would only create mismatches if a profile's semester drifted out of sync with the seeded data.

3. The controller's day logic

`load()` checks the logged-in profile: if teacher, fetch by teacher code; else fetch by batch+section. It defaults the selected day to today.

- **7-day week** - the university teaches Sun-Sat. `DateTime.weekday` (Mon=1..Sun=7) is mapped to short day names via a constant table, so today is highlighted correctly.
- **entriesForSelectedDay** - filters the loaded entries to the selected day's full name (e.g. 'Sunday').
- **countForDay** - powers the little count badges on each day chip.
- **setDay()** - changes the selected day and `notifyListeners()`; the screen rebuilds via `ListenableBuilder`.

4. RoutineEntry - the model that does the time math

Each row carries day, start/end time (as 'HH:MM' text), subject, teacher (name + code as TEXT), room, batch, section, semester. The smart part is the helper methods:

method	what it returns
<code>startOn(date)</code> <code>endOn(date)</code>	/ Builds a real <code>DateTime</code> for this slot on a given calendar day - so we can compare against `now`.
<code>timeLabel</code> / <code>startLabel</code>	Pretty 12-hour strings like '9:30 AM - 10:50 AM'.
<code>teacherDisplay</code>	Name if present, else code, else 'TBA'.
<code>statusOn(date, isToday)</code>	Returns <code>live</code> / <code>next</code> / <code>done</code> / <code>upcoming</code> for today's classes (delegates to <code>AppDateUtils.getClassStatus</code>).

`getClassStatus(start,end)`: now after end -> **done**; now after start -> **live**; starts within 30 min -> **next**; otherwise **upcoming**. This same logic powers the dashboards, Find Teacher and Rooms.

5. Likely defense questions

question	answer
How does one screen serve both roles?	The controller reads the profile's role and picks fetchForTeacher vs fetchForStudent; the UI is identical.
Where does the data come	The `routines` table - the same rows the timetable engine publishes.
Why store time as text?	It matches the engine output and DB column; the model parses it into DateTime on demand for comparisons.
How is 'today' computed for 7 days?	DateTime.weekday mapped through AppConstants.weekDays (Sun-Sat), the single source of truth shared with the engine.